

同济大学计算机系

计算机组成原理实验报告



学 号 2351520

姓 名 毛星博

专 业 计算机科学与技术

授课老师 陈永生

实验一：Fibonacci 数列

一、实验内容

Fibonacci 数列：将寄存器\$2,\$3 初始化为 fibonacci 数列的前两个数 0,1；寄存器\$4 为数列中所需得到的数字的序号（\$4=4 即表示得到第四个 Fibonacci 数）；最后得到的结果存入寄存器\$1。

二、MIPS 汇编程序代码

```
.text
main:
    # 初始化 F0 和 F1，设置项数 n
    addi $2,$zero,0
    addi $3,$zero,1
    addi $4,$zero,10

    #如果 n=0
    beq $4,$zero,case_zero

    #如果 n=1
    addi $s0,$zero,1
    beq $4,$s0,case_one

    #else
    addi $t1,$4,-1 #t1 寄存器中存放需要计算的次数，初始化为 n-1
    j fib_loop

fib_loop:
    add $t2,$2,$3 #计算下一项的值，临时存放在 t2
    addi $2,$3,0 #寄存器 3 挪到 2
    addi $3,$t2,0 #寄存器 t2 挪到 3
    addi $t1,$t1,-1 #计数器递减

    slt $t3,$zero,$t1 #如果$t1>0，$t3 置 1，否则置 0
    bne $t3,$zero,fib_loop

    add $1,$3,0
    j end_program

case_zero:
    #n=0 的情况
```

```

    addi $1,$zero,0
j end_program

```

```

case_one:
    #n=1 的情况
    addi $1,$3,0
j end_program

```

```

end_program:
    addi $v0, $zero, 10
    syscall

```

三、实验结果

1.特殊情况：n=0 或 n=1

```

main:
    # 初始化F0和F1，设置项数n
    addi $2,$zero,0
    addi $3,$zero,1
    addi $4,$zero,0

```

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x0000000a
\$v1	3	0x00000001
\$a0	4	0x00000000

(\$1) = 0

```

main:
    # 初始化F0和F1，设置项数n
    addi $2,$zero,0
    addi $3,$zero,1
    addi $4,$zero,1

```

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000001
\$v0	2	0x0000000a
\$v1	3	0x00000001
\$a0	4	0x00000001

(\$1) = 1

2.正常情况：n=4，n=10

```

main:
    # 初始化F0和F1，设置项数n
    addi $2,$zero,0
    addi $3,$zero,1
    addi $4,$zero,4

```

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000003
\$v0	2	0x0000000a
\$v1	3	0x00000003
\$a0	4	0x00000004

(\$1) = 3

```
main:
    # 初始化F0和F1，设置项数n
    addi $2, $zero, 0
    addi $3, $zero, 1
    addi $4, $zero, 10
```

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000037
\$v0	2	0x0000000a
\$v1	3	0x00000037
\$a0	4	0x0000000a

(**\$1**) = 55

实验二：冒泡排序

一、实验内容

将一串数列输入寄存器\$2-\$6,用冒泡排序算法对其进行排序

二、MIPS 汇编程序代码

```
.text
main:
    # $2-$6 存放排序的数据
    addi $2, $zero, 5
    addi $3, $zero, 3
    addi $4, $zero, 8
    addi $5, $zero, 1
    addi $6, $zero, 7

# 第一轮比较
round1_cmp1:
    slt $t0, $3, $2 # 如果 $3 < $2, 则 $t0=1
    beq $t0, $zero, round1_cmp2 # 如果不需要交换, 跳过
    # $3 >= $2, 交换二者的值
    add $t1, $3, $zero
    add $3, $2, $zero
    add $2, $t1, $zero

round1_cmp2:
    slt $t0, $4, $3
    beq $t0, $zero, round1_cmp3
    add $t1, $4, $zero
    add $4, $3, $zero
    add $3, $t1, $zero

round1_cmp3:
    slt $t0, $5, $4
    beq $t0, $zero, round1_cmp4
    add $t1, $5, $zero
    add $5, $4, $zero
    add $4, $t1, $zero

round1_cmp4:
    slt $t0, $6, $5
```

```
    beq $t0, $zero, round2_cmp1
    add $t1, $6, $zero
    add $6, $5, $zero
    add $5, $t1, $zero
```

第二轮比较

```
round2_cmp1:
    slt $t0, $3, $2
    beq $t0, $zero, round2_cmp2
    add $t1, $3, $zero
    add $3, $2, $zero
    add $2, $t1, $zero
```

```
round2_cmp2:
    slt $t0, $4, $3
    beq $t0, $zero, round2_cmp3
    add $t1, $4, $zero
    add $4, $3, $zero
    add $3, $t1, $zero
```

```
round2_cmp3:
    slt $t0, $5, $4
    beq $t0, $zero, round3_cmp1
    add $t1, $5, $zero
    add $5, $4, $zero
    add $4, $t1, $zero
```

第三轮比较

```
round3_cmp1:
    slt $t0, $3, $2
    beq $t0, $zero, round3_cmp2
    add $t1, $3, $zero
    add $3, $2, $zero
    add $2, $t1, $zero
```

```
round3_cmp2:
    slt $t0, $4, $3
    beq $t0, $zero, round4_cmp1
    add $t1, $4, $zero
    add $4, $3, $zero
    add $3, $t1, $zero
```

第四轮比较

```
round4_cmp1:
```

```

slt $t0, $3, $2
beq $t0, $zero, end_program
add $t1, $3, $zero
add $3, $2, $zero
add $2, $t1, $zero

```

```

end_program:
    #addi $v0, $zero, 10
    #syscall

```

三、实验结果

1.乱序数列 {5,3,8,1,7}

```

main:
    # $2-$6存放排序的数据
    addi $2, $zero, 5
    addi $3, $zero, 3
    addi $4, $zero, 8
    addi $5, $zero, 1
    addi $6, $zero, 7

```

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000001
\$v1	3	0x00000003
\$a0	4	0x00000005
\$a1	5	0x00000007
\$a2	6	0x00000008
\$a3	7	0x00000000

2.正序数列 {1,3,5,7,8}

```

main:
    # $2-$6存放排序的数据
    addi $2, $zero, 1
    addi $3, $zero, 3
    addi $4, $zero, 5
    addi $5, $zero, 7
    addi $6, $zero, 8

```

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000001
\$v1	3	0x00000003
\$a0	4	0x00000005
\$a1	5	0x00000007
\$a2	6	0x00000008
\$a3	7	0x00000000

3.倒序数列 {8,7,5,3,1}

```
main:
    #2-6存放排序的数据
    addi $2, $zero, 8
    addi $3, $zero, 7
    addi $4, $zero, 5
    addi $5, $zero, 3
    addi $6, $zero, 1
```

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000001
\$v1	3	0x00000003
\$a0	4	0x00000005
\$a1	5	0x00000007
\$a2	6	0x00000008
\$a3	7	0x00000000

4.有重复元素 {5,3,8,3,7}

```
main:
    #2-6存放排序的数据
    addi $2, $zero, 5
    addi $3, $zero, 3
    addi $4, $zero, 8
    addi $5, $zero, 3
    addi $6, $zero, 7
```

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000003
\$v1	3	0x00000003
\$a0	4	0x00000005
\$a1	5	0x00000007
\$a2	6	0x00000008
\$a3	7	0x00000000

5.全相同元素 {6,6,6,6,6}

```
main:
    #2-6存放排序的数据
    addi $2, $zero, 6
    addi $3, $zero, 6
    addi $4, $zero, 6
    addi $5, $zero, 6
    addi $6, $zero, 6
```

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000006
\$v1	3	0x00000006
\$a0	4	0x00000006
\$a1	5	0x00000006
\$a2	6	0x00000006
\$a3	7	0x00000000

实验三：布斯乘法算法

一、实验内容

运用布斯乘法算法实现两个数的乘法，结果用两个寄存器表示

二、MIPS 汇编程序代码

```
.text
main:
    # 初始化数据
    addi $t1, $zero, 128    # 被乘数
    addi $t2, $zero, -2     # 乘数

    # 检查符号并记录
    slt $t8, $t1, $zero     # 如果被乘数<0, 则$t8=1
    slt $t9, $t2, $zero     # 如果乘数<0, 则$t9=1
    xor $s7, $t8, $t9      # 结果符号: $s7=1 表示负, 0 表示正

    # 取绝对值
    bge $t1, $zero, skip_abs1
    sub $t1, $zero, $t1     # 如果被乘数<0, 取反
skip_abs1:
    bge $t2, $zero, skip_abs2
    sub $t2, $zero, $t2     # 如果乘数<0, 取反
skip_abs2:

    # 初始化乘法
    addi $t3, $zero, 0      # 部分积初始化为 0
    addi $t4, $zero, 0      # 附加位初始化为 0
    addi $t5, $zero, 32     # 计数器 (32 位乘法)

loop:
    # 检查最低两位 ($t2[0]和附加位$t4)
    andi $t6, $t2, 1        # 取乘数最低位到$t6
    sll $t6, $t6, 1         # 左移 1 位, 腾出空间给附加位
    or $t6, $t6, $t4        # 合并附加位到$t6 的低位

    # 根据条件分支
    beq $t6, $zero, shift   # 00: 仅右移
    addi $t7, $zero, 3
    beq $t6, $t7, shift     # 11: 仅右移
```

```

    addi $t7, $zero, 1
    beq $t6, $t7, add_op      # 01: 加被乘数
    addi $t7, $zero, 2
    beq $t6, $t7, sub_op      # 10: 减被乘数

add_op:
    add $t3, $t3, $t1          # 部分积 += 被乘数
    j shift

sub_op:
    sub $t3, $t3, $t1          # 部分积 -= 被乘数

shift:
    # 保存乘数最低位到附加位
    andi $t4, $t2, 1           # $t4 = $t2[0]

    # 右移部分积，并提取其最低位
    andi $t0, $t3, 1           # 保存部分积最低位到$t0
    sra $t3, $t3, 1            # 部分积算术右移 1 位

    # 右移乘数，并将部分积的最低位插入其最高位
    srl $t2, $t2, 1            # 乘数逻辑右移 1 位
    sll $t0, $t0, 31           # 将部分积的最低位左移到最高位
    or $t2, $t2, $t0           # 合并到乘数

    # 更新计数器
    addi $t5, $t5, -1
    bne $t5, $zero, loop

    # 根据符号调整结果（结果在$t3:$t2 中）
    beq $s7, $zero, end_program
    # 负结果：取补码
    nor $t3, $t3, $zero         # 先取反
    nor $t2, $t2, $zero         # 先取反
    addi $t2, $t2, 1            # 低位加 1
    beq $t2, $zero, carry       # 如果低位为 0，说明有进位
    j end_program

carry:
    addi $t3, $t3, 1            # 高位加 1

end_program:
    # 最终结果在$t3:$t2 中

    # 程序结束

```

```
addi $v0, $zero, 10
syscall
```

三、实验结果

1. 0*128

```
main:
    # 初始化数据
    addi $t1, $zero, 0    # 被乘数
    addi $t2, $zero, 128  # 乘数
```

\$t2	10	0x00000000
\$t3	11	0x00000000

2. 8*128

```
main:
    # 初始化数据
    addi $t1, $zero, 8    # 被乘数
    addi $t2, $zero, 128  # 乘数
```

\$t2	10	0x00000400
\$t3	11	0x00000000

3. -8*128

```
main:
    # 初始化数据
    addi $t1, $zero, -8   # 被乘数
    addi $t2, $zero, 128  # 乘数
```

\$t2	10	0xfffffc00
\$t3	11	0xffffffff

4. 8*-128

```
main:
    # 初始化数据
    addi $t1, $zero, 8    # 被乘数
    addi $t2, $zero, -128 # 乘数
```

\$t2	10	0xffffffffc00
\$t3	11	0xfffffffffff

5. -8*-128

```
main:
    # 初始化数据
    addi $t1, $zero, -8    # 被乘数
    addi $t2, $zero, -128  # 乘数
```

\$t2	10	0x00000400
\$t3	11	0x00000000

6. 大数相乘 131072*131072= (00000004 00000000) 16

```
main:
    # 初始化数据
    addi $t1, $zero, 131072    # 被乘数
    addi $t2, $zero, 131072    # 乘数
```

\$t2	10	0x00000000
\$t3	11	0x00000004